Chapter 3

Kernels, Beginning of neural networks

Reading

1. Bishop 6.1-6.3
2. Goodfellow 6.1-6.4
3. "Random features for large-scale kernel machines" by Rahimi and Recht (2008).

3.1 Digging deeper into the perceptron

3.1.1 Convergence rate

How many iterations does a perceptron need to fit on a given dataset? We will assume that the training data are bounded, i.e., $\|x^i\| \leq R$ for some R and for all $i \in \{1, \ldots, n\}$. Let us also assume that the training dataset is indeed linearly separable, i.e., a solution w^* exists for the perceptron weights with training error exactly zero. This means

$$y^i {w^*}^\top x^i > 0 \quad \forall i.$$

We will also assume that this classifier *separates the data well*. Note that the distance of each input x^i from the decision boundary (i.e., all x such that ${w^*}^\top x = 0$) is given by the component of x^i in the direction of w^* if the label is $y^* = +1$ and in the direction $-w^*$ if the label is negative. In other words,

$$\frac{y^i {w^*}^\top x^i}{\|w^*\|} = \rho^i$$

❓ Draw a picture to convince yourself that the distance to the boundary is indeed ρ.

gives the distance to the decision boundary. The quantity on the right hand side is called the *margin*, it is simply the distance of the sample i from the decision boundary. If w^* is the classifier with the largest average margin,

$$\rho = \min_{i \in \{1,\ldots,n\}} \rho^i$$

is a good measure of how hard a particular machine learning problem is.

You can now prove that after each update of the perceptron the inner product of the current weights with the true solution $\langle w_t, w^* \rangle$ increases at least linearly and that the squared norm $\|w_t\|^2$ increases at most linearly in the number of updates t. Together, the two will give you a result that after t weight updates

$$t \leq \frac{R^2}{\rho^2} \tag{3.1}$$

all training data are classified correctly. Notice a few things about this expression.

1. The quantity $\frac{R^2}{\rho^2}$ is dimension independent; that the number of steps to reach a given accuracy is independent of the dimension of the data. This will be a property shared by optimization algorithms in general.

2. There are no constant factors, this is also the worst case number of updates; this is quite rare and we cannot get similar results usually.

3. We can think of the quantity R^2/ρ^2 as a measure of the difficult of the problem. The number of updates scales with the difficulty; if the margin ρ were small, we need lots of updates to drive the training error to zero.

4. This formula also provides some insight into generalization. Let us make two assumptions (a) both train and test data have bounded inputs $\|x\| \leq R$, (b) there exists some weights w^* and margin ρ such that $y{w^*}^\top x \geq \rho \|w^*\|$ for all x, y. In short, we are assuming that the data can be classified perfectly by some perceptron. Now imagine a different training procedure that does not reuse any data, i.e., the perceptron looks at each sample only once, updates the weights and then throws away that sample after that iteration. The number of mistakes that the perceptron makes before it starts classifying *every* new datum correctly is also exact R^2/ρ^2. In other words, if the perceptron algorithm gets $n \sim R^2/\rho^2$ samples for a problem where the two assumptions hold, then it achieves perfect generalization.

3.1.2 Dual representation

Let us see how the parameters of the perceptron look after training on the entire dataset. At each iteration, the weights are updated in the direction (x^t, y^t) or they are not updated at all. Therefore, if α^i is the number of times the perceptron sampled the datum (x^i, y^i) during the course of its

⚠ If the perceptron makes a mistake in the tth update,

$$\begin{aligned} \langle w_t, w^* \rangle &= \langle w_{t-1} + y^{\omega_t} x^{\omega_t}, w^* \rangle \\ &\geq \langle w_{t-1}, w^* \rangle + \rho \|w^*\| \\ \implies \langle w^*, w_t \rangle &\geq t\rho \|w^*\|. \end{aligned}$$

Now,

$$\begin{aligned} &\|w_t\|^2 \\ &= \|w_{t-1} + y^{\omega_t} x^{\omega_t}\|^2 \\ &= \|w_{t-1}\|^2 + 2 \langle w_t, y^{\omega_t} x^{\omega_t} \rangle + \|x^{\omega_t}\|^2 \\ &\leq \|w_{t-1}\|^2 + \|x^{\omega_t}\|^2 \\ &\leq \|w_{t-1}\|^2 + R^2. \\ &\implies \|w_t\|^2 \leq tR^2. \end{aligned}$$

The cosine distance between w_t and w^* is

$$\begin{aligned} 1 \geq \cos(w_t, w^*) &= \frac{\langle w_t, w^* \rangle}{\|w^*\| \|w_t\|} \\ &\geq \frac{t\rho}{R\sqrt{t}} \\ \implies t &\leq \frac{R^2}{\rho^2}. \end{aligned}$$

training and got it wrong, we can write the weights of the perceptron as

$$w^* = \sum_{i=1}^{n} \alpha^i y^i x^i + w^{(0)}. \tag{3.2}$$

where $\alpha^i \in \{0, 1, \ldots, \}$ and $w^{(0)}$ is the initial weight configuration of the perceptron. Let us assume that $w^{(0)} = 0$ for the following discussion. The perceptron therefore is using the classifier

$$\begin{aligned} f(x, w) &= \text{sign}(\hat{y}) \\ \text{where} \quad \hat{y} &= \left(\sum_{i=1}^{n} \alpha^i y^i x^i \right)^\top x \\ &= \sum_{i=1}^{n} \alpha^i y^i {x^i}^\top x. \end{aligned} \tag{3.3}$$

As you see in Eq. (3.3), computing the prediction for a new input x involves, either remembering all the weights w at the end of training, or storing all the $\left\{\alpha^i\right\}_{i=1,\ldots,n}$ along with the training dataset. The latter is called the dual representation of a perceptron and the scalars $\left\{\alpha^i\right\}$ are called the dual parameters.

Remember this special form: the inner product of the new input x with all the other inputs x^i in the training dataset is combined linearly to get the prediction. The weights of this linear combination are the dual variables which measure of how many tries it took the perceptron to fit that sample during training.

3.2 Creating nonlinear classifiers from linear ones

Linear classifiers such as the perceptron, or the support vector machine (SVM) can be extended to nonlinear ones. The trick is essentially the same that we use when we fit polynomials (polynomials are nonlinear) using the formula for linear regression. We are interested in mapping input data x to some different space, this is (usually) a higher-dimensional space called the *feature space*.

$$x \mapsto \phi(x).$$

The quantity $\phi(x)$ is called a feature vector.

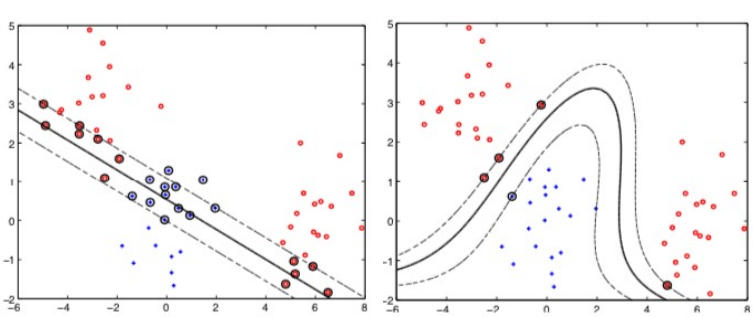


Figure 3.1

For example, in the polynomial regression case for scalar input data $x \in \mathbb{R}$ we used

$$\phi(x) := \left[1, \sqrt{2}x, x^2\right]^\top$$

to get a quadratic feature space. The role of $\sqrt{2}$ will become clear shortly. Certainly, this trick of creating polynomial features also works for higher dimensional input

$$\phi(x) := \left[1, x_1, x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2\right]^\top.$$

Having fixed a feature vector $\phi(x)$, we can now fit a linear perceptron on the input data $\left\{\phi(x^i), y^i\right\}$. This involves updating the weights at each iteration as

$$w^{(t+1)} = \begin{cases} w^{(t)} + y^t\phi(x^t) & \text{if sign}({w^{(t)}}^\top\phi(x^t)) \neq y^t \\ w^{(t)} & \text{else.} \end{cases} \tag{3.4}$$

At the end of such training, the perceptron is

$$w^* = \sum_{i=1}^n \alpha^i y^i \phi(x^i)$$

and predictions are made by first mapping the new input to our feature space

$$f(x; w) = \text{sign}\left(\sum_{i=1}^n \alpha^i y^i \phi(x^i)^\top \phi(x)\right). \tag{3.5}$$

Notice that we now have a linear combination of the *features* $\phi(x^i)$, not the data x^i, in our formula to compute the output.

❓ The concept of a feature space seems like a panacea. If we have complex data, we simply map it to some high-dimensional feature and fit a linear function to these features. However, the "curse of dimensionality" coined by Richard Bellman states that to fit a function in $\mathbb{R}^d$ the number of samples needs to be exponential in d. It therefore stands to reason that we need a lot more data to fit a classifier in feature space than in the original input space. Why would we still be interested in the feature space then?

3.3 Kernels

Observe the expression of the classifier in Eq. (3.5). Each time we make predictions on the new input, we need to compute n inner products of the form

$$\phi(x^i)^\top\phi(x).$$

If the feature dimension is high, we need to enumerate the large number of feature dimensions if we are using the weights of the perceptron, or these inner products if we are using the dual variables. Observe however that even if the feature vector is large, we can compactly evaluate the inner product

$$\begin{aligned} \phi(x) &= \left[1, \sqrt{2}x, x^2\right] \\ \phi(x') &= \left[1, \sqrt{2}x', {x'}^2\right] \\ \phi(x)^\top\phi(x') &= 1 + 2xx' + (xx')^2 = (1 + xx')^2. \end{aligned}$$

⚠ Feature spaces can become large very quickly. What is the dimensionality of $\phi(x)$ for a tenth-order polynomial with a three-dimensional input data? It is the number of terms in the multinomial expansion $(1 + x_1 + x_2 + x_3)^{10}$ which is 286.

for input $x \in \mathbb{R}$. Kernels are a formalization of this idea. A kernel

$$k : X \times X \to \mathbb{R}.$$

is any symmetric, positive semi-definite function with two arguments such that

$$k(x, x') = \phi(x)^\top \phi(x)$$

for some feature ϕ for all x, x'. Few examples of kernels are

$$k(x, x') = \left(x^\top x' + c\right)^2,$$
$$k(x, x') = \exp\left(-\|x - x'\|^2/(2\sigma^2)\right).$$

3.3.1 Kernel perceptron

We can now give the kernel version of the perceptron algorithm. The idea is to simply replace any inner product in the algorithm that looks like $\phi(x)^\top \phi(x')$ by the kernel $k(x, x')$.

Kernel perceptron Initialize dual variables $\alpha^i = 0$ for all $i \in \{1, \ldots, n\}$. Perform the following steps for iterations $t = 1, 2, \ldots$.

1. At the tth iteration, sample a data point with index ω_t from D_{train} uniformly randomly, call it $(x^{\omega_t}, y^{\omega_t})$.
2. If there is a mistake, i.e., if

$$\begin{aligned} 0 &\geq y^{\omega_t} \left(\sum_{i=1}^n \alpha^i y^i \phi(x^i)^\top \phi(x^{\omega_t}) \right) \\ &= y^{\omega_t} \left(\sum_{i=1}^n \alpha^i y^i k(x^i, x^{\omega_t}) \right), \end{aligned}$$

then update

$$\alpha^{\omega_t} \leftarrow \alpha^{\omega_t} + 1.$$

❓ Kernels look great, e.g., you can fit perceptrons in powerful feature spaces using essentially the same algorithm. How expensive is each iteration of the perceptron?

⚠ When ML algorithms are implemented in a system, there exist tradeoffs between the feature-space version and the Gram matrix version of linear classifiers. The former is preferable if the number of samples in the dataset is large, while the latter is used when the dimensionality of features is large.

Notice that we do not ever compute $\phi(x)$ so it does not matter what the dimensionality of the feature vector is. It can even be infinite, e.g., for the radial basis function kernel. Observe also that we do not maintain weights w. We instead maintain the dual variables $\left\{\alpha^1, \ldots, \alpha^n\right\}$ while running the algorithm.

Note that the kernel perceptron computes the kernel over *all* data samples in the training set at each iteration. It is expensive and seems wasteful. The Gram matrix denoted by $G \in \mathbb{R}^{n \times n}$

$$G_{ij} = k(x^i, x^j) \tag{3.6}$$

helps address this problem by computing the kernel on all pairs in the

❓ Logistic regression with a loss function

$$\ell_{\text{logistic}}(w) = \log\left(1 + e^{-yw^\top x}\right)$$

is also a linear classifier. Write down how you will fit a logistic regression using stochastic gradient descent; this is similar to the perceptron algorithm. Write down the feature-space version of the algorithm and a kernelized logistic regression that uses the Gram matrix.

training dataset. We can now write step 2 in the kernel perceptron

$$y^{\omega_t}\left(\sum_{i=1}^{n}\alpha^i y^i k(x^i, x^{\omega_t})\right) = y^{\omega_t}(\alpha \odot Y)^\top G e_{\omega_t}.$$

where $e_{\omega_t} = [0, \ldots, 0, 1, 0, \ldots]$ with a 1 on the ${\omega_t}^{\text{th}}$ element, $\alpha = \left[\alpha^1, \ldots, \alpha^n\right]$ denotes the vector of all the dual variables, $Y = \left[y^1, \ldots, y^n\right]$ is a vector of all the labels, and the notation $\alpha \odot Y = \left[\alpha^1 y^1, \ldots, \alpha^n y^n\right]$ denotes the element-wise (Hadamard) product. This expression now only involves a matrix-vector multiplication, which is more convenient than computing the kernel at each iteration. Gram matrices can become very big. If the number of samples is $n = 10^6$, not an unusual number today, the Gram matrix has 10^{12} elements. The big failing of kernel methods is that they require a large amount of memory at training time. Nystrom methods compute low-rank approximations of the Gram matrix which makes operations with kernels easier.

3.3.2 Mercer's theorem

This theorem shows that any kernel that satisfies some regularity properties can be rewritten as an inner product in some feature space.

A function $f : X \to \mathbb{R}$ is square integrable iff

$$\int_{x \in X} |f(x)|^2 \, \mathrm{d}x < \infty.$$

Theorem 3.1 (Mercer's Theorem). For any symmetric function $k : X \times X \to \mathbb{R}$ which is square integrable in $X \times X$ and satisfies

$$\int_{X \times X} k(x, x')\, f(x)\, f(x')\, \mathrm{d}x\, \mathrm{d}x' \geq 0 \tag{3.7}$$

for all square integrable functions $f \in L_2(X)$, there exist functions $\phi_i : X \to \mathbb{R}$ and numbers $\lambda_i \geq 0$ where

$$k(x, x') = \sum_{i=1}^{\infty} \lambda_i \phi_i^\top(x)\, \phi_i(x')$$

for all $x, x' \in X$. The condition in Eq. (3.7) is called Mercer's condition. You will also have seen Mercer's condition written as follows: "for any finite set of inputs $\left\{x^1, \ldots, x^n\right\}$ and any choice of real-valued coefficients $c_1, \ldots, c_n$ a valid kernel should satisfy

$$\sum_{i,j} c_i c_j k(x^i, x^j) \geq 0".$$

There can be an infinite number of coefficients λ_i in the summation.

We can think of a function $f(x)$ as a long vector with one entry for each $x \in X$. The integral in Theorem 3.1 in Mercer's condition is analogous to a vector-matrix-vector multiplication like $u^\top G u$.

There can be an infinite number of coefficients λ_i in the summation.

Remark 3.2 (Checking if a function is a valid kernel). Note that Mercer's condition states that the Gram matrix of any dataset is positive semi-definite:

$$u^\top G u \geq 0 \quad \text{for all } u \in \mathbb{R}^n. \tag{3.8}$$

This is easy to show.

$$
\begin{aligned}
u^\top G u &= \sum_{ij=1}^{n} u_i u_j G_{ij} \\
&= \sum_{ij} u_i u_j \left(\sum_{k=1}^{\infty} \lambda_k \phi_k(x^i)^\top \phi_k(x^j) \right) \\
&= \sum_{k=1}^{\infty} \lambda_k \left(\sum_{ij} u_i u_j \phi_k(x^i)^\top \phi_k(x^j) \right) \\
&= \sum_{k=1}^{\infty} \lambda_k \left(\sum_{i} u_i \phi_k(x^i) \right)^\top \left(\sum_{j} u_j \phi_k(x^j) \right) \\
&= \sum_{k=1}^{\infty} \lambda_k \left\| \sum_{i} u_i \phi_k(x^i) \right\|^2 \\
&\geq 0.
\end{aligned}
$$

On the second line, we have expanded the term $G_{ij} = k(x^i, x^j) = \sum_k \lambda_k \phi_k(x^i)^\top \phi_k(x^j)$ using Mercer's condition. So if you have a function that you would like to use as a kernel, checking its validity is easy by showing that the Gram matrix is positive semi-definite.

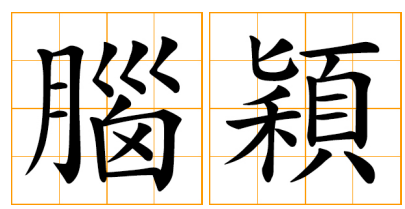
Kernels are powerful because they do not require you to think of the feature and parameter spaces. For instance, we may wish to design a machine learning algorithm for spam detection that takes in a variable length of feature vector depending on the particular input. If $x[i]$ is the ith character of a string, a good way to build a feature vector is to consider the set of all length k sub-sequences. The number of components in this feature vector is exponential. However, as you can imagine, given two strings x, x'

```
this string is interesting
txws sbhtqg is atso iyubqtnhpqg
```

you can write a Python function to check their similarities with respect to some rules *you define*, e.g., a small edit distance between the strings. Mercer's theorem is useful here because it says that so long as your function satisfies the properties of a kernel function, there exists some feature space which your Python function implicitly constructs.

❓ Checking your Python function for whether it is a good kernel is great using Eq. (3.8). Can you think of a situation when you can get a wrong answer using this approach, i.e., your kernel is not a legitimate kernel but Eq. (3.8) says that it is?

❓ These are two different images of related concepts, what feature space can we use to say that they are similar?



3.4 Learning the feature vector

> The central idea behind deep learning is to learn the feature vectors ϕ instead of choosing them *a priori*.

How do we choose what set of feature vectors to learn from? For instance, we could pick all polynomials; we could pick all possible Gabor filters

that you saw in HW 1; we could also pick all possible string kernels.

3.4.1 Random features

Suppose that we have a finite-dimensional feature $\phi(x) \in \mathbb{R}^p$. We saw in the perceptron that

$$f(x; w) = \text{sign}\left(\sum_i w_i \phi_i(x)\right)$$

where $\phi(x) = [\phi_1(x), \ldots, \phi_p(x)]$ and $w = [w_1, \ldots, w_p]$ are the feature and weight vectors respectively. We will set

$$\phi(x) = \sigma\left(S^\top x\right), \tag{3.9}$$

where $S \in \mathbb{R}^{d \times p}$ is a matrix. The function $\sigma(\cdot)$ is a nonlinear function of its argument and acts on all elements of the argument element-wise

$$\sigma(z) = [\sigma(z_1), \ldots, \sigma(z_p)]^\top.$$

We will abuse notation and denote both the vector version of σ and the element-wise version of σ using the same Greek letter.

Notice that this is a special type of feature vector (or a special type of kernel), it is a linear combination of the input elements. What matrix S should we pick to combine these input elements? The paper by Rahimi and Recht (2008) proposed the idea that for shift-invariant kernels (which have the property that $k(x, x') \equiv k(x - x')$) one may use a matrix with random elements as our S

$$S^\top = \begin{bmatrix} \omega_1^\top \\ \vdots \\ \omega_p^\top \end{bmatrix}$$

where $\omega_i \in \mathbb{R}^d$ are random variables drawn from, say, a Gaussian distribution and

$$\sigma(z) = \cos(z).$$

Using a random matrix is a cheap trick, it lets us create a lot of features quickly without worrying about their quality. Our classifier is now

$$f(x; w) = \text{sign}\left(w^\top \sigma\left(S^\top x\right)\right) \tag{3.10}$$

and we can again solve the optimization problem

$$w^* = \underset{w}{\text{argmin}} \frac{1}{n} \sum_{i=1}^{n} \ell_{\text{hinge}}(y^i, \hat{y}^i; w) \tag{3.11}$$

with $\hat{y^i} = w^\top \sigma\left(S^\top x^i\right)$ and fit the weights w using SGD as before.

Figure 3.2

As an example consider the heatmap of Gabor-like kernel $k(x, x')$ in Fig. 3.2 on the left. Each row and column corresponds to one particular input, x^i or x^j, so regions in the heatmap which are warm are pairs (x^i, x^j) that are similar under the kernel. We can think of the decomposition

$$\begin{aligned}
&\text{pixel } (i,j) \text{ of the left-most picture} \\
&= k(x^i, x^j) \\
&= \phi(x^i)^\top \phi(x^j) \\
&= \sum_{k=1}^{p} \sigma\left(\omega_k^\top x^i\right) \sigma\left(\omega_k^\top x^j\right) \qquad \text{(say for } p \text{ black-white matrices)} \\
&= \text{pixel } (i,j) \text{ in the right-most picture.}
\end{aligned}$$

In other words, the p random elements of the matrix S, namely ω_k come together to give us a useful kernel on the left. A large random matrix S has many such terms on the right hand-side.

3.4.2 Learning the feature matrix as well

Random features do not work well for all kinds of data. For instance, if you have an image of size 100×100, and you are trying to find a fruit

we can design random features of the form

$$\phi_{ij,kl} = \mathbf{1}_{\{\text{mostly red color in a box formed by pixels } (ij) \text{ and} (kl)\}}.$$

We will need lots and lots of such features before we can design an object detector that works well for this image. In other words, random features do not solve the problem that you need to be clever about picking your feature space/kernel.

❓ What kind of data do you think random features will work well for?

Simply speaking, deep learning is about learning the matrix S

in Eq. (3.10) in addition to the coefficients w. The classifier now is

$$f(x; w, S) = \text{sign}\left(w^\top \sigma\left(S^\top x\right)\right) \tag{3.12}$$

but we now solve the optimization problem

$$w^*, S^* = \underset{w,S}{\text{argmin}} \frac{1}{n} \sum_{i=1}^{n} \ell_{\text{hinge}}(y^i, \hat{y}^i) \tag{3.13}$$

with $\hat{y}^i = w^\top \sigma\left(S^\top x^i\right)$ as before. This is our first *deep network*, Eq. (3.12) is a two-layer neural network.

Moving from the problem in Eq. (3.11) to this new problem in Eq. (3.13) is a very big change.

1. **Nonlinearity.** The classifier in Eq. (3.12) is not linear anymore. It is a nonlinear function of its parameters w, S (both of which we will call weights).

2. **High-dimensionality.** We added a lot more weights to the classifier, the original classifier had $w \in \mathbb{R}^p$ parameters to learn while the new one also has $S \in \mathbb{R}^{d \times p}$ more weights. The curse of dimensionality suggests that we will need a lot more data to fit the new classifier.

3. **Non-convex optimization.** The optimization problem in Eq. (3.13) much harder than the one in Eq. (3.11). The latter is a convex function (we will discuss this soon) which are easy to minimize. The former is a non-convex function in its parameters w, S because they interact multiplicatively, such functions are harder to minimize. We could write down the solution of the perceptron using the final values of the dual variables. We cannot do this for a two-layer neural network.

Bibliography

Rahimi, A. and Recht, B. (2008). Random features for large-scale kernel machines. In *Advances in neural information processing systems*, pages 1177–1184.